
TradeNet Fabric

A comprehensive Q&A; guide covering networking fundamentals, Python programming, systems infrastructure, and project-specific questions. Every concept explained from first principles — no prior networking knowledge assumed.

29 tests passing	1.84ms p99 path comp	2.88ms p99 failover
8 property tests	3 chaos scenarios	6 architecture decisions
12 devices · 3 vendors	4 security zones	314 auto-gen firewall rules

Contents

- Section 1: Routing Protocols (Q1–Q6)
- Section 2: Python & Programming (Q7–Q9)
- Section 3: Systems & Infrastructure (Q10–Q12)
- Section 4: Project-Specific Questions (Q13–Q16)
- Section 5: Bonus Questions (Q17–Q30)
- Quick Reference — Key Numbers

Section 1: Routing Protocols

Q1. What happens at the packet level when an OSPF adjacency forms?

What OSPF is: A program running on routers that figures out how to get data from A to B. It talks to neighboring routers and builds a complete map of the network — like Google Maps for your network.

An adjacency is a relationship between two directly connected routers. They go through a 7-step introduction:

Step 1 — Down:

Router just booted. Knows nothing. Like moving into a new neighborhood.

Step 2 — Init:

Sends a Hello packet to multicast address 224.0.0.5: 'I'm Router A, here's my ID, I'm in Area 1.'

Step 3 — Two-Way:

Neighbor replies with a Hello containing A's name. Both know communication works both ways. Like exchanging phone numbers.

Step 4 — ExStart:

Negotiate who talks first (higher ID = master). Agree on message numbering.

Step 5 — Exchange:

Swap tables of contents of their map databases (DBD packets). Not full details yet.

Step 6 — Loading:

Each requests full details (LSAs) for anything it's missing.

Step 7 — Full:

Both have identical maps. Each independently runs Dijkstra's algorithm to compute fastest routes.

Interview tip: Default hello/dead timers are 10s/40s — too slow for trading. We use 1s/4s with BFD (300ms failure detection) — 133x faster than defaults.

Q2. Why does BGP use TCP while OSPF uses raw IP?

IP = base layer. Says 'from X, to Y' but doesn't guarantee delivery. **TCP** = layer on top that adds reliability (handshake, ordering, retransmission). **Raw IP** = skip TCP, just fire and hope.

OSPF uses raw IP because:

- Only talks to directly connected neighbors (one hop). Low loss risk.
- Has its own acknowledgment system (LSAcks).
- Uses multicast for neighbor discovery — TCP can't do multicast.
- Lower overhead — important when sending Hellos every 1 second.

BGP uses TCP because:

- Peers can be many hops apart (DC-East to DC-West across the country).
- Updates can be huge (real internet: 1M+ routes). TCP handles large transfers.
- Sessions last days/weeks. TCP built for long-lived connections.

Interview tip: 'OSPF is like shouting to your neighbor. BGP is like mailing a package cross-country — you need a reliable postal service (TCP).'

Q3. Link-state vs distance-vector protocols?

Distance-vector (e.g., RIP):

Each router only knows what neighbors told it. Like road signs: 'NYC: 200 miles.' Slow convergence.
Count-to-infinity problem — failures can take MINUTES to propagate.

Link-state (e.g., OSPF — what we use):

Every router floods its link info to ALL routers. Everyone has the same complete map. Each runs Dijkstra independently. Like everyone having GPS. Convergence under 1 second.

We chose OSPF because trading firms need sub-second failover. Distance-vector is unacceptable.

Q4. How would you debug a routing loop?

A **routing loop**: packets bounce between routers forever. Router A sends to B, B sends back to A, repeat infinitely.

Step 1:

Confirm with traceroute — if same IPs repeat, you have a loop.

Step 2:

Check routing tables on both routers. If each points to the other for the same destination — that's your loop.

Step 3:

Identify which protocol wrote the bad entry (O=OSPF, B=BGP, S=static).

Step 4:

OSPF: Check if both have identical link-state databases. MTU mismatch is a common sneaky cause.

BGP: Check next-hop reachability. BGP may install a route with unreachable next-hop.

Redistribution: OSPF→BGP→OSPF feedback loops need prefix filters.

Interview tip: Our property tests verify 'no path visits the same node twice' across 1,600 random topologies — loops are mathematically impossible in our path computation.

Q5. Explain the BGP path selection algorithm.

When a router knows about the SAME destination from multiple sources, BGP uses ordered tiebreakers. First difference wins:

- 1. Highest Weight** — Local sticky note: 'always prefer this.' Only one router sees it.
- 2. Highest Local Preference** — Shared across your org. THE main knob our SDN controller uses.
- 3. Locally originated** — Prefer routes this router created itself.
- 4. Shortest AS-Path** — Fewer hops through organizations = preferred. Weak hijack defense.
- 5. Lowest Origin type** — IGP > EGP > Incomplete. Minor tiebreaker.
- 6. Lowest MED** — Neighbor's suggestion: 'prefer this link to me.' SDN controller also uses this.
- 7. eBGP over iBGP** — External routes preferred over internal.
- 8. Lowest IGP metric** — 'Hot-potato routing' — hand off at nearest exit.
- 9-10. Oldest route, lowest Router ID** — Final tiebreakers for stability.

Interview tip: 'Our SDN controller uses Local Preference (#2) and MED (#6) to dynamically steer traffic based on live latency — traditional networks set these once and forget.'

Q6. What is BFD and why is it critical for trading?

BFD (Bidirectional Forwarding Detection) = tiny fast heartbeat. Two routers exchange 'I'm alive' every 100ms. 3 missed = 300ms = link declared dead.

Without BFD:

Link fails → 4s to detect (OSPF dead timer) → SPF runs → ~5s total.

With BFD:

Link fails → 300ms to detect → OSPF notified → SPF runs → ~500ms total.

BFD is 13x faster than aggressive OSPF timers. 4 seconds of blackholed traffic = thousands of missed price updates = lost money.

Section 2: Python & Programming

Q7. Why Python for the SDN controller, and how do you keep it safe?

The controller ingests telemetry, computes paths, and serves a REST API. Python optimizes for development speed and ecosystem; we buy back the safety a compiler would give us with disciplined design and tests.

1. One language across the whole stack:

The controller, chaos framework, benchmarks, and automation are all Python — no FFI, no context-switching, shared tooling and data models.

2. Immutable data for safe concurrency:

The graph is built from frozen dataclasses; every update returns a NEW graph. The telemetry collector and path engine read consistent snapshots without locks — we just swap the reference atomically.

3. Safety via enums, type hints, and tests:

link_state is an enum (Up, Down, Degraded); records are typed dataclasses. We catch the bugs a compiler would with type hints (mypy-checkable) and a strong suite — 29 tests including property-based checks across 1,600 random topologies.

Interview tip: 'Python trades compile-time guarantees for velocity and ecosystem. For a controller that pushes routing changes to live routers, we compensate with immutable dataclasses, enums, and property-based tests that prove invariants hold — not just that one example works.'

Q8. What is Dijkstra's algorithm?

Finds the cheapest path from one point to all others in a weighted graph.

Analogy — finding shortest route to the airport:

1. Start at home. Distance to self = 0. Everything else = infinity.
2. Look at all places reachable directly. Record distances.
3. Go to closest unvisited place. (Key: this IS the shortest path to it.)
4. From there, check neighbors. If going through you is shorter, update them.
5. Repeat until destination reached.

Our key design: parameterized weight function.

Pass a FUNCTION defining 'cost.' Market data: cost = latency. Bulk transfer: cost = inverse bandwidth. Same algorithm, different objectives.

Performance: 1.84ms p99 for our 12-node topology. $O((V+E) \log V)$ complexity.

Q9. What are property-based tests?

Unit tests: 'This specific input gives this specific output.' One scenario.

Property tests: 'For ANY connected graph, a path exists between any two nodes.' Computer generates hundreds of random inputs and checks.

Our 8 properties:

1. Connected nodes always have a path
2. Shortest path is actually shortest (no cheaper alternative)
3. $A \rightarrow B$ costs the same as $B \rightarrow A$

4. No path visits the same node twice (no loops)
5. After killing any link, surviving paths still loop-free
6. Making a link slower never makes path through it cheaper
7. JSON roundtrip preserves graph exactly
8. Market data path always has lowest latency

Interview tip: Jane Street uses property-based testing extensively. Our pytest suite verifies 8 invariants across 1,600 random topologies (built by a seeded generator).

Section 3: Systems & Infrastructure

Q10. What is eBPF and why use it over sFlow/NetFlow?

OS has two layers: **userspace** (normal programs, sandboxed) and **kernel** (core, manages hardware). Network packets handled by kernel.

eBPF runs a small verified program INSIDE the kernel. No copying packets out. BPF verifier proves safety before loading (no infinite loops, no null pointers).

sFlow:

Samples 1 in 1000 packets. Misses 99.9%. A 50µs jitter spike? Probably missed.

NetFlow:

Aggregates into 60-second summaries. Way too slow. Averages away spikes.

eBPF:

Every packet. Nanosecond timestamps. Custom logic. Minimal overhead. Trading-grade.

Q11. What is RPKI and how does it prevent BGP hijacks?

Every network has an **AS number** (license plate) and owns IP ranges. BGP has NO authentication — anyone can claim 'I can route to NYSE.'

RPKI adds cryptographic proof. NYSE publishes a signed ROA: 'Only AS 11111 may announce 198.51.100.0/24.' Our routers check every announcement against these records.

VALID — ROA exists, ASN matches. Accept.

INVALID — ROA exists, ASN doesn't match. HIJACK. Reject.

NOT FOUND — No ROA. Accept cautiously.

We tested: AS99999 announced NYSE's prefix. RPKI caught it immediately, marked INVALID, generated alert.

Q12. Route reflectors vs full iBGP mesh?

iBGP rule: Can't re-advertise routes learned from one iBGP peer to another. Prevents loops but requires full mesh.

Full mesh: $N \text{ routers} = N(N-1)/2 \text{ sessions}$. 4 routers = 6 sessions. 50 routers = 1,225. Doesn't scale.

Route reflectors are ALLOWED to re-advertise. Central mail room instead of everyone mailing everyone. We use TWO RRs for redundancy.

Section 4: Project-Specific Questions

Q13. Why both Ansible AND Nornir?

Ansible = recipe book (declarative):

'Every Cisco router should have this exact config.' Push templates to 24 devices.

Nornir = doctor doing rounds (programmatic):

'Check each router's OSPF neighbors, compare to expected, alert if wrong.' Real Python logic.

Ansible's YAML logic is painful for complex conditionals. Nornir's Python is clean. Using both shows engineering taste.

Q14. Why zone-based firewall instead of ACLs?

ACLs are stateless — each packet judged independently. Allow outbound? You ALSO need separate inbound rule for return traffic. Miss it? Silently dropped.

Zone-based are stateful — track connections. Allow outbound, returns auto-permitted. Zones are architectural: 'trading talks to exchange' maps to business requirements.

4 zones: Trading (90), Exchange (50), Management (100), Internet (0). Auto-generated 314 lines of Junos config from YAML.

Q15. Walk through a link failure in your system.

0ms — Fiber breaks.

100ms — First missed BFD heartbeat.

200ms — Second missed.

300ms — Third missed. BFD declares link DEAD. Notifies OSPF.

310ms — OSPF floods new LSA to all routers.

400ms — All routers run Dijkstra. New routes computed.

500ms — Traffic rerouted. Failover complete.

Meanwhile — SDN controller detects, runs Dijkstra (1.84ms p99), pushes BGP changes.

Total SDN failover: 2.88ms p99. Target 100ms. We're 35x under.

Q16. Why EVE-NG over GNS3 or Containerlab?

EVE-NG: Web UI, clean multi-vendor support, industry standard for professional labbing.

GNS3: Client-server model adds complexity. Multi-vendor clunkier.

Containerlab: Modern but not all vendors have good container images.

UTM over VirtualBox: Free, native Apple Silicon. VBox has poor M-series support.

Section 5: Bonus Questions

Q17. What is ARP and why does it matter?

ARP (Address Resolution Protocol) translates IP addresses to MAC addresses. IP = mailing address. MAC = name on the mailbox. Your computer broadcasts 'Who has 10.1.0.1?' The device replies with its MAC. Result cached.

ARP only works within one network segment. To reach another subnet, you ARP for your default gateway (router), not the final destination.

Q18. TCP vs UDP?

TCP:

Reliable, ordered, connection-oriented. 3-way handshake. Every byte acknowledged. Like a phone call.

UDP:

Unreliable, unordered, connectionless. Just fire packets. Like throwing a letter over a fence.

Market data from exchanges is typically UDP multicast. Missing one packet is better than waiting for retransmission and getting stale data late.

Q19. What is subnetting?

IP = 32 bits. **Subnetting** divides into network portion + host portion. 10.1.0.0/24 = first 24 bits fixed (network), last 8 bits for hosts (256 addresses).

/32 for loopbacks (one address), /30 for point-to-point links (2 usable), /24 for server subnets (254 usable).

Our scheme: 10.{site}.{function}.{host} — predictable and debuggable.

Q20. What are OSPF areas and why?

Areas = groups of routers sharing detailed maps. Within your area, you know every street. Between areas, only major highways.

- Limit blast radius: flap in Area 1 doesn't affect Area 2
- Reduce memory and CPU per router
- Our project: each site = own area (DC-East=0.0.0.1, Colo-NYSE=0.0.0.4)

Q21. What is a BGP community?

A **tag** attached to a route — colored sticky note. Doesn't change the route, just labels it for policy decisions.

Tag as 'learned-from-nyse' → write policy: 'prefer routes tagged learned-from-nyse.'

Tag as 'do-not-export' → prevent internal routes leaking externally.

Communities scale BGP policy. Instead of rules per-prefix (thousands), tag at the edge, write rules per-community (handful).

Q22. Why use loopback interfaces for BGP?

Loopback = virtual interface, always up as long as router runs. Physical interfaces die when a cable is cut.

BGP sessions on loopbacks survive individual link failures as long as ANY path exists between routers. OSPF ensures loopbacks are reachable via any available path.

Q23. What is traffic engineering?

Controlling which path traffic takes, rather than letting protocols decide alone. OSPF picks shortest by cost — but 'shortest' isn't always 'best' (maybe it's congested).

Our SDN controller:

1. Collects live metrics from all links
2. Runs Dijkstra with live weights (not static costs)
3. Adjusts BGP attributes to steer traffic when optimal path changes

Traditional: set costs once, forget. Ours: continuously optimized. This IS the value of SDN.

Q24. iBGP vs eBGP?

eBGP: Between DIFFERENT organizations. NYSE (AS 11111) to us (AS 65001). Different AS numbers.

iBGP: WITHIN same organization. DC-East to DC-West. Same AS number.

eBGP: peers 1 hop apart, modifies AS-path. iBGP: peers many hops apart (loopbacks), does NOT modify AS-path (hence full-mesh/RR requirement).

Q25. What is MTU and why problems?

MTU = biggest packet size a link can carry. Standard: 1500 bytes. Jumbo: 9000 bytes.

If Router A sends 9000-byte packet but link MTU is 1500, packet is fragmented or dropped.

OSPF trap: If two sides disagree on MTU, adjacency gets STUCK in ExStart. Most common OSPF troubleshooting issue.

Trading: Jumbo frames reduce overhead, but ONE mismatched link degrades performance.

Q26. NETCONF vs SSH/CLI for automation?

SSH/CLI: type commands, parse text output with regex. Fragile — if output format changes, parser breaks.

NETCONF: structured XML, transactional (commit/rollback), candidate config (stage changes, review diff, then apply), vendor-neutral protocol.

Our SDN controller would use NETCONF to push BGP changes — structured, transactional, rollback-safe.

Q27. What is leaf-spine topology?

Traditional 3-tier (core/distribution/access) creates bottlenecks. **Leaf-spine:** every leaf connects to EVERY spine. Always exactly 2 hops. Predictable latency, no bottleneck, easy to scale.

Our project: DC-East and DC-West use Arista in leaf-spine. Cisco core connects to WAN.

Q28. What does 'convergence' mean?

All routers agreeing on the same routes after a change. Until complete, different routers have different views — packets may take unexpected paths or get dropped.

OSPF: <1s with BFD. BGP: 5-30s. Our SDN controller: 2.88ms p99.

During convergence, traffic may be blackholed. Shorter = fewer dropped packets = fewer missed market updates.

Q29. Multicast vs unicast?

Unicast: One sender, one receiver. 1000 viewers = 1000 copies sent. Inefficient.

Multicast: One sender, many receivers. ONE copy sent, network replicates to all subscribers.

NYSE sends ONE market data stream (239.1.1.1). Network copies to every trading firm. PIM-SM builds the distribution tree.

Q30. If you could redesign one part, what would you change?

(Tests self-awareness and engineering maturity.)

Suggested answer:

'I'd add gRPC streaming telemetry — push-based instead of pull-based. Right now the controller polls via REST. In production, devices should continuously stream metrics, reducing detection delay to near-zero.'

'I'd also move the API to async I/O — e.g. FastAPI/asyncio — so telemetry ingestion and path computation don't block each other under load. Shows awareness of where the Python stack is heading.'

This shows you understand the limitation, know the production solution, AND track where the Python ecosystem is heading. All strong signals.

Quick Reference — Key Numbers

Metric	Value	Context
Path Computation P99	1.84ms	Target: 50ms (27x under)
Failover P99	2.88ms	Target: 100ms (35x under)
API Health P99	1.46ms	Target: 10ms
Unit Tests	21 passing	pytest framework
Property Tests	8 properties	1,600 random topologies
Total Tests	29 passing	0 failures
Devices	12	Cisco / Arista / Juniper
Links	12	All up, live metrics
Sites	6	3 DCs + 3 exchange colos
Security Zones	4	Trading, Exchange, Mgmt, Internet
Chaos Scenarios	3	All passing
ADRs	6	Documented design decisions
Firewall Rules	314 lines	Auto-generated from YAML
RPKI Hijacks Blocked	2	AS99999 + AS88888 rejected
BFD Detection	300ms	vs 40s default (133x faster)
OSPF Hello/Dead	1s / 4s	vs 10s / 40s default
BGP Timers	10s / 30s	vs 60s / 180s default
Internal ASN	65001	All sites, route reflectors

Good luck, Mo.